
Práctica 3: Pasadas y efectos

Fecha de entrega: 10 de febrero de 2026, 15:40

Objetivo: Pasadas y efectos de profundidad y alpha clipping

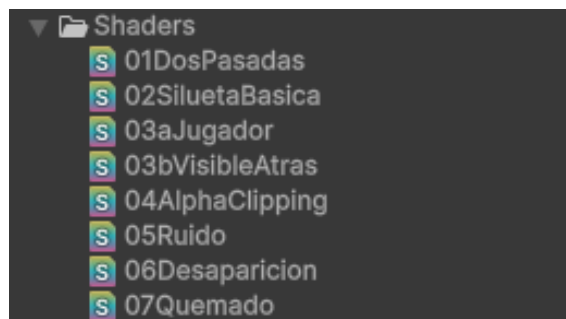
1. Instrucciones

[Iguales a las de la práctica anterior]

La práctica se realiza sobre un proyecto de Unity en el que se van añadiendo shaders y materiales que se aplican sobre un cubo.

La entrega debe incluir únicamente los shaders individuales con los nombres *de ficheros* concretos indicados en el texto y una captura de la pestaña *Game* del editor de Unity en la que se vea el cubo con el material aplicado. Es importante que los nombres de los ficheros entregados coincidan con los pedidos. En el enunciado también se especifica qué nombre deben tener las variables HLSL de los parámetros del material.

Si vas creando los shaders en la carpeta *Shaders* del proyecto, al final de la práctica deberías ver algo así:



En la entrega habrá un fichero con extensión `.shader` por cada uno de los archivos que aparecen en la imagen, así como una captura del resultado de aplicar ese material con extensión `.png` o `.jpg`. Por ejemplo, se espera un fichero `01DosPasadas.shader` y un `01DosPasadas.png` (o `.jpg`).

Para el desarrollo se recomienda crear la escena base con el cubo y duplicarla de forma que en cada escena el cubo utilice un material y shader distinto.

2. Punto de partida

[Igual al de la práctica anterior]

Crea un proyecto 3D en Unity que utilice el *Universal Render Pipeline* con una escena con:

- El contenido por defecto de la plantilla de Unity, con skybox, cámara y luz direccional.
- La cámara con la localización (L), escalado (S) y rotación (R) siguientes: L: $(-3, 1, -3)$, S: $(1, 1, 1)$, R: $(10, 45, 0)$.
- Un cubo en L: $(0, 0, 0)$, S: $(1, 1, 1)$, R: $(0, 0, 0)$.

3. Dos pasadas

Los *render pipeline* (RP) dividen la construcción del frame en varias etapas. La geometría de la escena puede, por tanto, pintarse varias veces utilizando distintos VS y PS. Para eso, en cada SubShader se puede añadir más de una *pasada* y, utilizando sus tags, indicar en qué etapa del RP se utilizará.

Para pruebas rápidas (como esta) se puede conseguir que la misma geometría se renderice dos veces en la fase de dibujado de opacos. Basta con especificar una segunda pasada sin añadir el tag definiendo el `LightMode`. Se puede añadir a cada pasada un nombre, que será utilizado en las ventanas de depuración del frame.

```
Shader "Unlit/NombreDelShader"
{
    Properties { [ ... ] }
    SubShader
    {
        Tags { [ ... ] }

        Pass
        {
            Name "Pasada1"
            Tags { "LightMode" = "UniversalForward" }

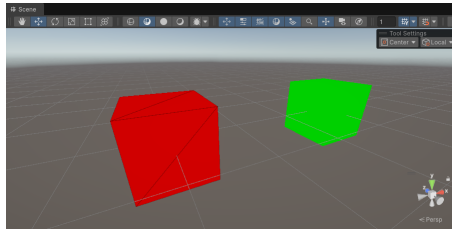
            [ ... ]
        }
        Pass
        {
            Name "Pasada2"

            // Sin tags, como mecanismo (feo...) para que URP
            // utilice ambas pasadas en la fase de dibujado de la escena
            //Tags { "LightMode" = "UniversalForward" }

            [ ... ]
        }
    }
}
```

```
}
```

Crea un *Unlit shader* simple llamado `01DosPasadas` que dibuje la geometría dos veces una en rojo y otra en verde. La pasada en verde debe simplemente desplazarla tres unidades en el eje X. Los nombres de cada pasada pueden ser `DibujadoEstandar` y `DibujadoTraslado`, por ejemplo.

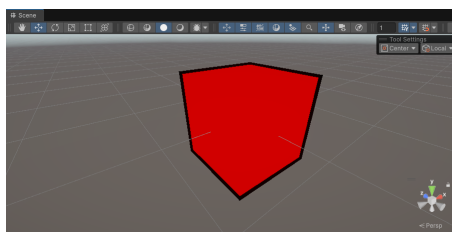


Observa que la segunda copia del cubo no “existe” para Unity:

- Si en la pestaña *Scene* se pincha sobre el cubo verde, el editor no lo selecciona.
- Si se configura la pestaña para que pinte también el *alambre* de la malla (como en la captura), únicamente aparece en el cubo rojo.
- Si se mueve la cámara para que el cubo rojo quede fuera de su ángulo de visión, Unity deja de pintar el objeto y el cubo verde desaparece.

4. Silueta básica

Crea un *Unlit shader* llamado `02SiluetaBasica` que, utilizando dos pasadas, dibuje la geometría de color rojo con una silueta de color negro.



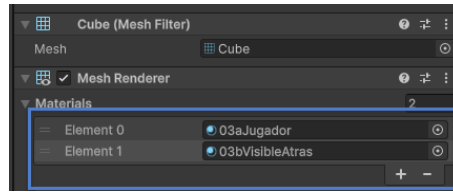
Para conseguirlo, la pasada de la silueta pinta en negro *las caras traseras* tras haber alejado todos sus vértices un 5 % en cada eje del origen de coordenadas del modelo (o lo que es lo mismo, multiplicando por 1.05).

5. Jugador siempre visible

Queremos que el “jugador” (en nuestra escena será el cubo) aparezca siempre visible, de forma que si hay algo delante de él, éste se dibuje en rojo.

Para poder probarlo, añade en la escena un plano en L: $(-1.5, 0, 0)$, R: $(-90, 45, 0)$ y S: $(0.2, 0.2, 0.2)$ de forma que tape parcialmente el cubo en la escena del juego.

Para hacer el efecto necesitarás crear dos shaders (y materiales), `03aJugador` (pinta con una textura configurable) y `03bVisibleAtras` (pinta color rojo plano) y asignar ambos al cubo:



Para que funcione, el shader rojo tiene que invertir la comprobación de profundidad (`ZTest Greater`) y no escribir en el Z-Buffer (`ZWrite Off`)¹

Recuerda que para que funcione, la geometría que utilice `03bVisibleAtras` debe pintarse después de todo lo opaco. Puedes forzarlo desde el shader cambiando el tag `"Queue" = "Geometry+10"`.

6. Alpha clipping

Crea un nuevo shader, `04AlphaClipping` para hacer un *alpha clipping* sencillo de un objeto (recuerda ajustar la prioridad con la que se ejecuta). El material debe admitir una textura que se utilizará para darle la apariencia al objeto. Tendrá, además, un parámetro real:

```
Properties
{
    // [... definición del parámetro para la textura ...]
    _CutThresold("Cutoff threshold", float) = 0.0
}
```

El PS descartará todos los píxeles que tengan, *en el espacio local del objeto* una coordenada Y por encima del parámetro `_CutThresold`. Para que el efecto no quede mal, configura el shader para pintar también las caras traseras (`Cull Off` en la pasada).

Para comprobar que funciona correctamente:

- Crea en la escena un segundo cubo (o una esfera) al lado del primero pero colocado en `Y=-0.5` y con el doble de escala.
- Con valores de 0.5 o más, ambos objetos deben verse completamente.
- Con un valor de 0 los cubos deben quedar cortados por la mitad (haz la captura con este valor).
- Con un valor de -0.5 debe desaparecer (quizá el cubo por problemas de precisión no termine de desaparecer por completo, sino que se vea parcialmente la cara inferior).

7. Peril Noise

El ruido es un recurso muy utilizado en efectos (sin ir más lejos, en el apartado siguiente) y puede venir en una textura o calcularse de forma procedimental en una función. El más famoso es el *Peril noise* por su importancia histórica al ser utilizado en una de las primeras películas en utilizar CGI (*Tron*, en 1982).

En ShaderGraph, el lenguaje visual de Unity para crear materiales, existe un nodo que recibe una coordenada de textura y una escala y ruido de este tipo (un valor entre 0 y 1).

¹En la escena de prueba esto no es relevante, al no haber más objetos. Piensa un ejemplo donde sí lo sería.

El código equivalente al nodo aparece a continuación (es fácil encontrarlo buscando algo como *shadergraph peril noise code*).

```
float2 unity_gradientNoise_dir(float2 p)
{
    p = p % 289;
    float x = (34 * p.x + 1) * p.x % 289 + p.y;
    x = (34 * x + 1) * x % 289;
    x = frac(x / 41) * 2 - 1;
    return normalize(float2(x - floor(x + 0.5), abs(x) - 0.5));
}

float unity_gradientNoise(float2 p)
{
    float2 ip = floor(p);
    float2 fp = frac(p);
    float d00 = dot(unity_gradientNoise_dir(ip), fp);
    float d01 = dot(unity_gradientNoise_dir(ip + float2(0, 1)), fp - float2(0, 1));
    float d10 = dot(unity_gradientNoise_dir(ip + float2(1, 0)), fp - float2(1, 0));
    float d11 = dot(unity_gradientNoise_dir(ip + float2(1, 1)), fp - float2(1, 1));
    fp = fp * fp * fp * (fp * (fp * 6 - 15) + 10);
    return lerp(lerp(d00, d01, fp.y), lerp(d10, d11, fp.y), fp.x);
}

void Unity_GradientNoise_float(float2 UV, float Scale, out float Out)
{
    Out = unity_gradientNoise(UV * Scale) + 0.5;
}
```

Crea un nuevo shader y material llamado 05Ruido que tenga un parámetro de escala:

```
Properties
{
    _NoiseScale("Noise Scale", float) = 20
}
```

y que incorpore en la sección de HLSL el código anterior y que, en lugar de utilizar una textura, pinte el cubo convirtiendo a gris el resultado de llamar a la función de ruido con las coordenadas de textura del punto y la escala del parámetro del material.

8. Desaparición

El efecto de desaparición con *alpha clipping* anterior generaba un corte demasiado repentino. Lo ideal sería que el parámetro `_CutThresold` definiera la altura del corte pero que éste fuera progresivo en una “banda” entre `_CutThresold-CutHeight` y `_CutThresold+CutHeight` (de esa forma, con un valor de 0 tendremos el efecto anterior con un “corte limpio”).

Combina los dos materiales anteriores en uno 06Desaparicion que utilice el ruido. La idea básica es que en cada punto se muestree el ruido y se añada a su coordenada Y antes de compararla con el umbral. De esa forma algunos puntos por encima del umbral aún se verán y otros por debajo no.

Como la función de ruido devuelve un valor entre 0 y 1 tendrás que escalarla a un valor entre -1 y 1. Antes de sumarselo a la Y, aplícale una escala en base al nuevo parámetro del material `_CutHeight`:

```
Properties
```

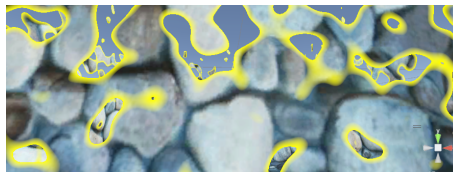
```
{  
    // [...]  
    _CutHeight("Cutoff height", Range(0.0, 1.0)) = 0.3  
}
```

9. Quemando el objeto

El último efecto 07Quemado consiste en ampliar el anterior para que los bordes que comienzan a desaparecer aparezcan de amarillo. En concreto, añade un último parámetro `_CutWidth` que indique el ancho (en coordenadas del objeto) de una *banda amarilla* que aparecerá alrededor de los cortes.

```
Properties  
{  
    // [...]  
    _CutWidth("Cut width", Range(0.0, 1)) = 0.1  
}
```

La banda tendrá un color amarillo puro en el borde y se irá interpolando con la textura de forma que a una distancia `_CutWidth` habrá desaparecido, simulando que el objeto es de papel y se está quemando.



10. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- Los ficheros de shaders y capturas indicados anteriormente con los nombres exactos.
- Incluir además un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.